MCP Drift Project: Status Briefing

Target: IEEE S&P 2027, cycle 2 · Prepared 2 Nov 2026 · Code: /home/user/mcp-drift

1. Where we are, in one paragraph

The data collection and first analysis are **done**. We crawled 4,812 MCP server repositories, walked the release history of 3,508 of them, and extracted 35,725 versions from the 2,571 servers that expose tools in at least two versions. First results show that servers frequently change after release, mostly in code rather than in what users see. **None of the numbers are validated yet.** The single thing blocking the paper is your manual labelling of ~260 sampled rows, which turns raw counts into precision-backed claims.

2. Deadlines

Date (23:59 AoE)	Milestone	Owner
Nov 4	Label first 20 rows of each sheet; send to me	You
Nov 6	Finish all labelling; ask a reviewer to read a draft Nov 14-15	You
Nov 10	Abstract registration on the S&P submission site	You (I draft)
Nov 14	Full draft ready for external reader	Me + you
Nov 17	Full paper submission	You (I prepare files)

Please confirm both dates on sp2027.ieee-security.org/cfpapers.html.

3. The research idea

Working title: *Trusted Once, Changed Later: A Longitudinal Study of Post-Approval Drift in MCP Servers*

Users and MCP clients approve a server's tools once. Every prior MCP security measurement (DCIChecker, MCPDiFF, MCP-FSD, MCPZoo) analyses a **single snapshot**. The Unicode/rug-pull paper (arXiv 2607.05744) shows clients force re-approval for 0 of 8 techniques but states it cannot estimate how often this happens in the wild. TrustShiftProbe studies malicious runtime changes only on synthetic servers. Longitudinal studies exist only for agent *skills* (Skill Constellations, Oct 2026). **Nobody has measured how real MCP servers change after approval.** That is our gap.

RQ	Question
RQ1 Prevalence	How often do servers gain capabilities (shell, code execution, file write, secret access, network, new domains) after their first release?
RQ2 Disclosure	How often is that gain silent: tool list, descriptions and README unchanged?
RQ3 Descriptions	How often do approved tool descriptions change, and does the change add instruction-like or hidden content?
RQ4 Detection (optional)	Can a diff-only detector flag security-relevant undisclosed changes?

4. Process: what was built

Step	What it does	Result
1. Crawl	GitHub search over topics mcp-server / model-context-protocol / mcp-servers, ≥20 stars, no forks or archives	4,812 repos; 3,508 in Python/TypeScript/JavaScript
2. Version selection	Release tags (up to 40) if a repo has ≥3; otherwise 12 commits spaced in time; latest commit always included	Median 8 versions per server
3. Tool extraction	tree-sitter parser finds tool definitions (decorators, add_tool, registerTool, tool(), Tool(), schema objects) with name and description	Tools per version
4. Capability detection	Patterns for shell, dynamic code, file read/write, env/secrets, network, external domains, in each tool's handler and helpers up to 4 calls deep. Records whether a capability is in the tool's own code ('self') or which helper adds it	Per-tool and per-server capabilities

Step	What it does	Result
5. Drift analysis	Compares consecutive versions; flags capability gains, description changes, suspicious text, new domains and dependencies; checks whether each gain was disclosed	events.jsonl, repo_summary.csv
6. Validation sampler	Seeded, stratified sample for hand-labelling, one event per repo per stratum	200 drift events + 60 tools

5. Results so far (UNVALIDATED)

2,571 servers, 35,725 versions. Percentages are shares of servers with at least one such event.

Finding	Servers	Comment
Tool description changed after first release	65.6%	Descriptions are not stable; approval-once is fragile
Existing tool gained high-risk capability, description unchanged (any path)	15.6%	Includes helper-reached gains; upper bound
Same, gain inside the tool's own handler	2.1%	Stricter; likely high precision
Server gained high-risk capability with no tool / description / README change	3.6%	Fully silent at server level
New external domain appeared	54.4%	Needs filtering of docs/CDN URLs
Description change matched 'suspicious' keywords	8.1%	All 14 concealment / hidden-character hits read by hand: benign

How to read this. The defensible claim is about code: servers change after approval, and those changes are rarely visible in what users approved. The realistic range for the headline is **roughly 2-16%** of servers; your labels decide where in that range it lands. We found **no evidence of malicious description rug pulls**; instruction-like text ("Do NOT use X, use this instead") is common and harmless, so keyword scanners cannot tell steering from injection. That is a reportable finding and an argument for checking code diffs.

Still to add: per-version and per-year rates (servers with more versions have more chances to change), figures, and precision numbers from your labels.

6. Problems found and fixed

Problem	Fix
Python decorator tools (@mcp.tool, the most common pattern) never had their handler code attached, so their capabilities were always empty	Fixed; full rerun of all repos with the corrected extractor
FastMCP server.add_tool(...) not recognised (e.g. Scrapling showed 0 tools)	Added support
Unclear where a capability came from (e.g. 'cursor_position' gaining 'shell' via a deep helper)	Each capability now records 'self' or the helper name; results reported both ways
Huge monorepos saturate server-level signals	Repos with >600 source files excluded (248 repos)
Background runs limited to 2 hours	Extraction split into 3 shards; all completed

7. What I need from you now

- **Label 20 + 20 rows today/tomorrow.** Open `validation/drift_events.csv` and `validation/extractor_tools.csv`; follow `validation/README.md`. For each drift event, open the compare link and choose: *extraction_error*, *benign*, *security_relevant*, or *unsure*. Do not look at my labels; we will compute agreement (Cohen's kappa), which reviewers expect.
- **Then label the remaining rows** (~200 events, 60 tools; about 3-4 hours total) by Nov 6.
- **Confirm the deadlines** on the S&P 2027 CFP page and read its rules on resubmission and anonymity.
- **Read three papers** so you can defend related work: DCIChecker (2606.04769), Unicode TAG / rug pull (2607.05744), TrustShiftProbe (2608.23763).
- **Line up a reader**: a professor or senior researcher who can read the draft on Nov 14-15.
- **Decide authorship and affiliation** details for the submission form (needed for abstract registration on Nov 10).

8. What I do next

- Per-version and per-year normalised rates; figures (prevalence by capability, disclosure breakdown, time to first gain).
- My own independent labels on the same first 20 rows, then the agreement score.
- Title and abstract draft for Nov 10, with precision numbers left blank until your labels are in.
- Paper draft: introduction, method, results, case studies, limitations, ethics and open-science sections.
- Optional RQ4 detector if time allows after the draft.

9. Known limitations (will be stated in the paper)

- Static pattern-based capability detection; capabilities inside third-party libraries are invisible.
- Helper resolution is by function name, which can match the wrong function; this is why validation matters.
- Only git history of the default branch; npm/PyPI packages that users install may differ.
- Repos with >600 source files excluded; star threshold of 20 biases toward popular servers.

10. Where everything is

Path	Contents
src/crawl.py, extract.py, history.py, analyze.py, sample.py	Full pipeline
data/repos.jsonl	Crawled repository list
data/snapshots/	Per-repo version snapshots (v2 extractor); data/snapshots_v1/ is the buggy first run
data/events.jsonl, data/repo_summary.csv	Drift events and per-server summary
validation/	Labelling sheets and instructions
notes/related_work.md, notes/plan.md	Positioning against prior work; plan and results log